Введение

Цель работы - реализовать алгоритм сортировки TIMSORT на языке Python, сгенерировать 100 наборов входных данных, измерить время работы и количество шагов, а затем сравнить практические результаты с теоретической сложностью.

Измерения выполнялись только для самой сортировки. Чтение данных из файла, генерация входных массивов и построение графиков в измеряемое время не включались.

1. Название алгоритма и краткая историческая справка

TIMSORT - гибридный устойчивый алгоритм сортировки, сочетающий идеи сортировки слиянием и сортировки вставками. Алгоритм был предложен Тимом Питерсом для Python. Его важная особенность состоит в том, что он умеет использовать уже имеющийся порядок в данных, поэтому хорошо работает не только на случайных массивах, но и на почти отсортированных последовательностях..

2. Описание принципа работы и особенностей TIMSORT

Основные идеи TIMSORT можно описать так:

- массив просматривается слева направо и в нём ищутся естественные серии - уже отсортированные подпоследовательности;
- если серия оказалась убывающей, она разворачивается, чтобы получить возрастающий run;
- слишком короткие серии дополняются и сортируются вставками до минимального размера min_run;
- после этого серии постепенно сливаются между собой, пока не получится один отсортированный массив.

Преимущества алгоритма: устойчивость, хорошая работа на реальных данных и адаптивность.

3. Описание программной реализации

Проект состоит из трёх основных Python-файлов: timsort.py, data_generator.py и benchmark.py.

Файл timsort.py содержит сам алгоритм сортировки и функции get_min_run, insertion_sort, merge, find_run и timsort. Файл data_generator.py создаёт 100 текстовых файлов со случайными массивами. Файл benchmark.py читает каждый файл, запускает сортировку, измеряет время работы и количество шагов, сохраняет таблицу результатов и строит графики.

4. Генерация входных данных

Было сгенерировано 100 наборов случайных входных данных. Размеры массивов изменяются от 100 до 10 000 элементов.

Каждый файл содержит один массив целых чисел.

Полные 100 файлов находятся в папке data/input проекта.

5. Способы измерения работы алгоритма

По условию работы были реализованы два способа измерения:

- по времени выполнения - функция measure_time запускает сортировку несколько раз и берёт медианное значение времени в миллисекундах;
- по количеству шагов - функция measure_steps запускает сортировку в режиме подсчёта и получает число сравнений, сдвигов и записей.

Под количеством шагов в данной работе понимается не количество машинных инструкций, а счётчик логических операций, который увеличивается при сравнении элементов, записи в массив и некоторых вспомогательных действиях внутри сортировки.

Таблицы результатов

краткая сводка по всем 100 экспериментам.

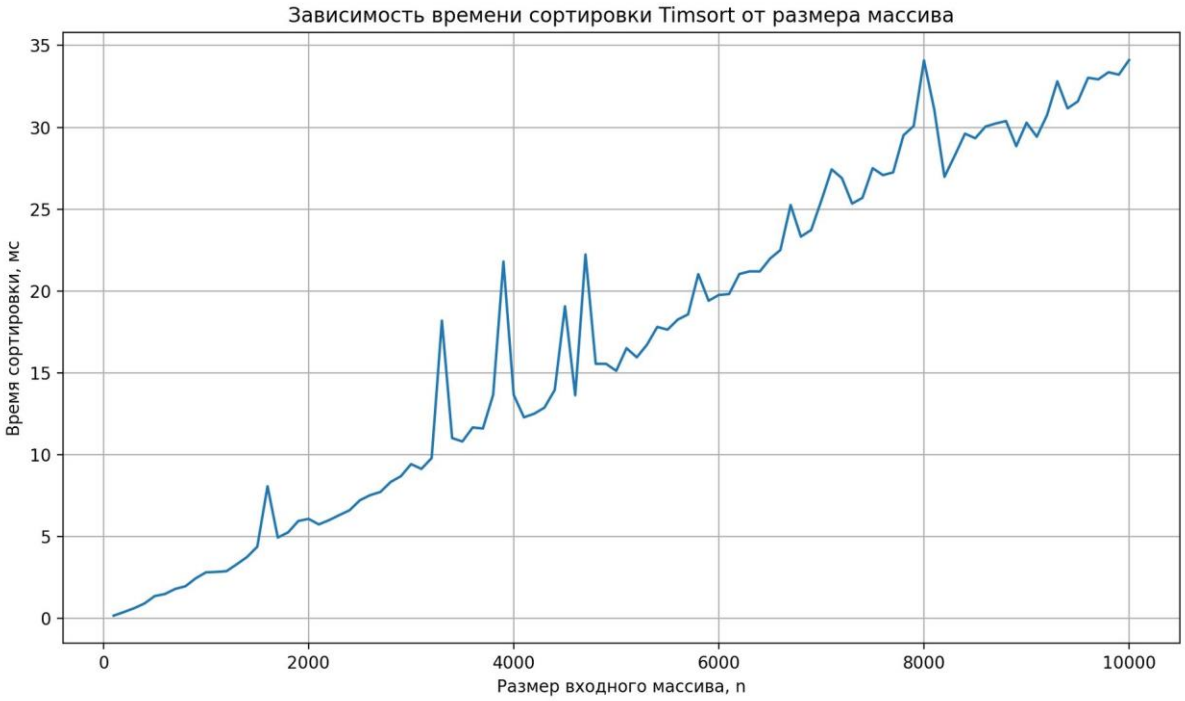
Показатель	Значение
Количество наборов данных	100
Минимальный размер массива	100
Максимальный размер массива	10000
Среднее время сортировки, мс	17.25
Минимальное время сортировки, мс	0.181232
Максимальное время сортировки, мс	34.106141
Среднее количество шагов	146256
Минимальное количество шагов	1936
Максимальное количество шагов	292719
Коэффициент корреляции size/time	0.9839
Коэффициент корреляции size/steps	0.9926

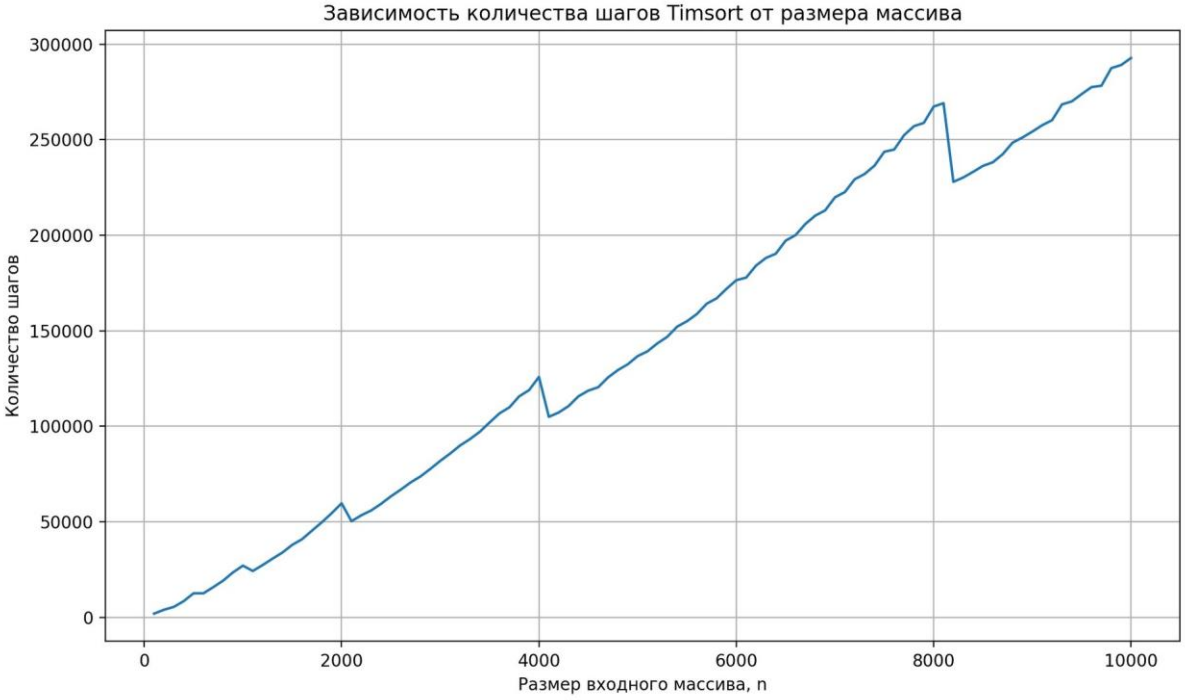
Далее приведён фрагмент основной таблицы измерений для некоторых характерных размеров входного массива.

№ набора	Размер n	Время, мс	Шаги
1	100	0.181232	1936
10	1000	2.824304	27038
20	2000	6.089559	59675
30	3000	9.433453	81882
40	4000	13.657633	125814
50	5000	15.12835	136695
60	6000	19.760037	176445
70	7000	25.531605	219830
80	8000	34.08404	267312
90	9000	30.278229	254224
100	10000	34.106141	292719

6. Графики

Ниже приведены графики зависимости времени выполнения и количества шагов от размера входного массива.





7. Оценка сложности алгоритма и сравнение теории с практикой

Теоретически TIMSORT относится к адаптивным устойчивым алгоритмам. Для него обычно указывают:

- лучший случай - O(n), если данные уже близки к отсортированным и состоят из длинных естественных серий; • средний случай - O(n log n);
- худший случай - O(n log n).

Для моей реализации по памяти можно считать O(n), потому что при слиянии создаются временные списки left_part и right_part.

Практические результаты на случайных массивах согласуются с теорией O(n log n). Коэффициент корреляции между размером массива и временем составил 0.9839, а между размером массива и количеством шагов - 0.9926. Отношение steps / (n log2 n) в среднем равно 2.342, меняясь в пределах примерно от 2.119 до 2.914. Это показывает, что рост числа шагов близок к функции n log n.

На графике времени есть небольшие скачки. Это нормально: на время влияют особенности интерпретатора Python, работа операционной системы и фоновые процессы. Но общий тренд роста остаётся близким к теоретическому.

8. Выводы

В ходе выполнения семестровой работы была реализован алгоритм TIMSORT на Python.

- создана программа генерации 100 входных файлов размером от 100 до 10 000 элементов;
- реализовано чтение массивов из файлов перед запуском сортировки;
- добавлены два способа измерения: по времени выполнения и по количеству шагов; • построены графики зависимости времени и шагов от размера входных данных;
- выполнено сравнение практических результатов с теоретической сложностью.

По итогам эксперимента можно сделать вывод, что реализованный алгоритм работает корректно и на случайных массивах демонстрирует рост трудоёмкости, близкий к O(n log n).

Список источников

1. Python Software Foundation. Sorting HOW TO. Официальная документация Python.
2. CPython source code. Objects/listsort.txt. Описание алгоритма timsort в исходных текстах CPython.
3. McIlroy P. Optimistic Sorting and Information Theoretic Complexity. Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, 1993.

Приложение А. Код алгоритма TIMSORT (timsort.py)

Ниже приведён полный код реализации алгоритма TIMSORT, использованной в данной работе.

```
MIN_RUN = 32


def get_min_run(n):
r = 0     while n >=
MIN_RUN:
        r |= n & 1
n >>= 1     return
n + r


# обычная сортировка вставками
# right включительно
# counter[0] хранит число шагов

def insertion_sort(arr, left, right,
counter):     for i in range(left + 1, right
+ 1):         temp = arr[i]         j = i - 1

        while j >= left:
counter[0] += 1  # сравнение
if arr[j] > temp:                 arr[j
+ 1] = arr[j]
counter[0] += 1  # сдвиг
                j -=
1             else:
break

        arr[j + 1] = temp
counter[0] += 1


# слияние двух отсортированных частей:
# left..mid и mid+1..right

def merge(arr, left, mid, right, counter):
left_part = arr[left:mid + 1]
right_part = arr[mid + 1:right + 1]

    i = 0
j = 0     k
= left

    while i < len(left_part) and j <
len(right_part):         counter[0] += 1  #
сравнение         if left_part[i] <= right_part[j]:
arr[k] = left_part[i]
i      += 1        else:             arr[k] = right_part[j]
j      += 1
        counter[0] += 1  # запись
k += 1

    while i < len(left_part):
arr[k] = left_part[i]
i      += 1        k
+= 1
counter[0] += 1

    while j < len(right_part):
arr[k] = right_part[j]
j      += 1        k
+= 1
counter[0] += 1
```

```
# поиск естественной серии (run)
# если серия убывающая, переворачиваем ее

def find_run(arr, start, n,
counter):     if start == n - 1:
return 1     run_end = start + 1

    counter[0] += 1     if
arr[run_end] < arr[start]:
while run_end < n:             if
run_end == start + 1:
run_end += 1
continue

            counter[0] += 1             if
arr[run_end] < arr[run_end - 1]:
run_end += 1             else:
break

        arr[start:run_end] = arr[start:run_end][::-1]
        counter[0] += (run_end -
start)     else:         while run_end
< n:             if run_end == start +
1:                 run_end += 1
continue

            counter[0] += 1             if
arr[run_end] >= arr[run_end - 1]:
run_end += 1             else:
break     return run_end - start


# упрощенная учебная версия timsort
# использует:
# 1) поиск естественных серий
# 2) сортировку вставками для коротких серий
# 3) попарное слияние серий

def timsort(arr,
count_steps=False):     n =
len(arr)     counter = [0]

    if n <= 1:
if count_steps:
return arr, 0
return arr, None

    min_run = get_min_run(n)
    runs = []
i = 0

    while i < n:         run_len =
find_run(arr, i, n, counter)

        if run_len < min_run:
right = i + min_run - 1
if right >= n:
right = n - 1
            insertion_sort(arr, i, right, counter)
run_len = right - i + 1

        runs.append((i, run_len))
i += run_len

    while len(runs) > 1:
new_runs = []
```

```
        i = 0

        while i < len(runs):
if i == len(runs) - 1:
new_runs.append(runs[i])
                i += 1             else:
left_start, left_len = runs[i]
right_start, right_len = runs[i + 1]

                left = left_start
                mid = left_start + left_len - 1
right = right_start + right_len - 1

                merge(arr, left, mid, right, counter)
                new_runs.append((left_start, left_len +
right_len))                 i += 2         runs = new_runs

    if count_steps:
return arr, counter[0]
return arr, None


if __name__ == '__main__':     a =
[5, 1, 9, 3, 7, 2, 8, 4, 6]
print('До сортировки:', a)
    timsort(a)
    print('После сортировки:', a)
```

Приложение Б. Код генерации входных данных (data_generator.py)

Ниже приведён код программы, которая создаёт 100 файлов со случайными массивами.

```
import os
import random


def generate_files(folder, count=100, min_size=100,
max_size=10000):     if not os.path.exists(folder):
os.makedirs(folder)     random.seed(42)

    if count == 1:         sizes = [min_size]
else:         step = (max_size - min_size) //
(count - 1)
        sizes = []         for i in
range(count - 1):
sizes.append(min_size + i * step)
sizes.append(max_size)

    for i in range(count):         size = sizes[i]
numbers = []         for _ in range(size):
numbers.append(random.randint(-100000, 100000))

        file_name = 'dataset_{:03d}_n{:05d}.txt'.format(i + 1, size)
path = os.path.join(folder, file_name)

        with open(path, 'w', encoding='utf-8') as f:
            f.write(' '.join(map(str, numbers)))
print('Файлы с входными данными созданы в папке:', folder)
```

```
if __name__ == '__main__':     current_dir =
os.path.dirname(os.path.abspath(__file__))
    project_dir = os.path.dirname(current_dir)
```

```
    data_folder = os.path.join(project_dir, 'data', 'input')
generate_files(data_folder)
```

Приложение B. Код программы измерения результатов (benchmark.py)

Этот файл отвечает за чтение входных массивов, запуск сортировки, подсчёт шагов, измерение времени и построение графиков.

```
import csv import math import os
import statistics import sys import
time import matplotlib.pyplot as plt

current_dir = os.path.dirname(os.path.abspath(__file__)) if
current_dir not in sys.path:     sys.path.append(current_dir)
from timsort import timsort


def read_numbers(file_path):     with open(file_path, 'r',
encoding='utf-8') as f:         text = f.read().split()
return [int(x) for x in text]


def measure_time(numbers, repeats=3):
times = []     for _ in range(repeats):
arr = numbers.copy()         start =
time.perf_counter()
        timsort(arr)
        end = time.perf_counter()
times.append((end - start) * 1000)     return
statistics.median(times)


def measure_steps(numbers):     arr
= numbers.copy()
    _, steps = timsort(arr, True)     return
steps


def save_table(rows, file_path):     with open(file_path, 'w',
newline='', encoding='utf-8') as f:         writer = csv.writer(f)
        writer.writerow(['dataset', 'file_name', 'size', 'time_ms', 'steps', 'n_log2_n'])
for row in rows:             writer.writerow(row)


def make_graph(sizes, values, ylabel, title, file_path):
plt.figure(figsize=(10, 6))     plt.plot(sizes, values)
    plt.xlabel('Размер входного массива, n')
    plt.ylabel(ylabel)
plt.title(title)     plt.grid(True)
```

```
    plt.tight_layout()
    plt.savefig(file_path, dpi=200)     plt.close()


def main():     project_dir =
os.path.dirname(current_dir)
    input_dir = os.path.join(project_dir, 'data', 'input')     results_dir
= os.path.join(project_dir, 'results')

    if not os.path.exists(results_dir):         os.makedirs(results_dir)     files =
sorted([name for name in os.listdir(input_dir) if name.endswith('.txt')])

    rows = []     sizes =
[]     times = []
steps_list = []

    for index, file_name in enumerate(files, start=1):
file_path = os.path.join(input_dir, file_name)
        numbers = read_numbers(file_path)
n = len(numbers)

        time_ms = measure_time(numbers)
steps = measure_steps(numbers)         n_log =
round(n * math.log2(n), 3)

        rows.append([index, file_name, n, round(time_ms, 6), steps, n_log])
sizes.append(n)         times.append(time_ms)         steps_list.append(steps)
print('Обработан файл:', file_name)     save_table(rows,
os.path.join(results_dir, 'benchmark_results.csv'))


make_graph(         sizes,
times,
        'Время сортировки, мс',
        'Зависимость времени сортировки Timsort от размера массива',
os.path.join(results_dir, 'time_vs_size.png')     )


make_graph(         sizes,
steps_list,
        'Количество шагов',
        'Зависимость количества шагов Timsort от размера массива',
os.path.join(results_dir, 'steps_vs_size.png')     )

    with open(os.path.join(results_dir, 'summary.txt'), 'w', encoding='utf-8') as f:
        f.write('Количество наборов данных: ' + str(len(rows)) + '\n')
        f.write('Минимальный размер массива: ' + str(min(sizes)) + '\n')
        f.write('Максимальный размер массива: ' + str(max(sizes)) + '\n')
        f.write('Минимальное время, мс: ' + str(round(min(times), 6)) + '\n')
        f.write('Максимальное время, мс: ' + str(round(max(times), 6)) + '\n')
        f.write('Минимальное количество шагов: ' + str(min(steps_list)) + '\n')
        f.write('Максимальное количество шагов: ' + str(max(steps_list)) + '\n')     print('Готово.

Результаты сохранены в папке results')


if __name__ == '__main__':     main()
```

Приложение Г. Приложение со входными данными

Так как полные входные данные занимают 100 отдельных файлов, в отчёте приведены их структура и фрагмент. Полный комплект входных данных находится в папке data/input архива проекта.

Пример фрагмента файла dataset_001_n00100.txt:

```
67621 -70816 -93444 94393 -27903 -35804 -41487 -63421 93061 -73132 77392 94161
42964 -77210 54794 10604 -91669 -92189 -75439 -42686 -39010 32475 57815 -93044
47127 -47876 87700 70363 83848 42853 9974 -42213 17757 54472 -27074 -98297
98916 -58147 83013 10785 -10806 -27158 -59242 -43557 -11764 -73207 -75688 -405 -
74647 -5896 -9835 58263 -30658 -88610 91294 20435 40568 -67278 -770 -79344
```

Список первых 10 файлов из набора входных данных:

```
dataset_001_n00100.txt
dataset_002_n00200.txt
dataset_003_n00300.txt
dataset_004_n00400.txt
dataset_005_n00500.txt
dataset_006_n00600.txt
dataset_007_n00700.txt
dataset_008_n00800.txt
dataset_009_n00900.txt
dataset_010_n01000.txt
```